

Advanced OOP Case Study: Food Ordering Application (Role-Based + Analytics)

Requirements:

- Use ALL OOP principles (Abstraction, Encapsulation, Inheritance, Polymorphism)
- Use ArrayList and Collections Framework
- Implement Custom Exceptions
- Use Functional Interface (Lambda)
- String validations
- Coupon Engine
- Order Cancellation with Penalty
- Sorting & Analytics Dashboard
- Students must fill missing logic (TODO sections)

Entities:

1. User (Abstract) -> Admin, Customer
2. FoodItem
3. Order

----- User.java -----

```
public abstract class User {

    protected int userId;
    protected String name;
    protected String role;

    public User(int userId, String name, String role) {

        if (userId <= 0)
            throw new IllegalArgumentException("Invalid ID");

        // TODO:
        // Validate name (min 3 characters)
        // Validate role not null

        this.userId = userId;
        this.name = name;
        this.role = role;
    }

    public abstract void displayRole();
}
```

----- Admin.java -----

```
public class Admin extends User {

    public Admin(int id, String name) {
        super(id, name, "ADMIN");
    }

    @Override
    public void displayRole() {
        System.out.println("Admin Access Granted");
    }
}
```

----- Customer.java -----

```
public class Customer extends User {

    private List<Order> orders = new ArrayList<>();

    public Customer(int id, String name) {
        super(id, name, "CUSTOMER");
    }

    public void addOrder(Order order) {
        // TODO: Add order to list
    }

    @Override
    public void displayRole() {
        System.out.println("Customer Access Granted");
    }
}
```

----- FoodItem.java -----

```
public class FoodItem {

    private int itemId;
    private String name;
    private double price;

    public FoodItem(int itemId, String name, double price) {

        // TODO:
        // Validate name not empty
        // Validate price > 0

        this.itemId = itemId;
        this.name = name;
        this.price = price;
    }
}
```

```

    }

    public double getPrice() {
        return price;
    }
}

----- Functional Interface -----

@FunctionalInterface
public interface CouponEngine {
    double applyDiscount(double totalAmount);
}

----- Order.java -----

public class Order {

    private int orderId;
    private Customer customer;
    private List<FoodItem> items = new ArrayList<>();
    private double totalAmount;
    private String status; // PLACED, CANCELLED

    public Order(int orderId, Customer customer) {
        this.orderId = orderId;
        this.customer = customer;
        this.status = "PLACED";
    }

    public void addItem(FoodItem item) {
        // TODO: Add item to list
    }

    public void calculateTotal() {
        // TODO:
        // Use Stream API to sum item prices
    }

    public void applyCoupon(CouponEngine engine) {
        // TODO:
        // Apply lambda discount
    }

    public void cancelOrder()
        throws OrderCancellationException {

        // TODO:
        // If already cancelled throw exception
        // Deduct 10% penalty
        // Change status
    }

    public double getTotalAmount() {
        return totalAmount;
    }

    public String getStatus() {
        return status;
    }
}

----- Exceptions -----

public class OrderCancellationException extends Exception {
    public OrderCancellationException(String message) {
        super(message);
    }
}

----- FoodService.java -----

public class FoodService {

    private List<Customer> customers = new ArrayList<>();
    private List<FoodItem> menu = new ArrayList<>();

```

```

private List<Order> orders = new ArrayList<>();

public void addCustomer(Customer c) {
    customers.add(c);
}

public void addFoodItem(FoodItem item) {
    menu.add(item);
}

public void placeOrder(Order order) {
    orders.add(order);
}

public void sortOrdersByAmount() {
    // TODO:
    // Sort using Collections.sort or Comparator
}

public double calculateTotalRevenue() {
    // TODO:
    // Stream API
    // Sum only PLACED orders
    return 0;
}

public void analyticsDashboard() {
    // TODO:
    // Print:
    // 1. Total Orders
    // 2. Total Revenue
    // 3. Highest Order Amount
}
}

----- MainApp.java -----

public class MainApp {

    public static void main(String[] args) {

        FoodService service = new FoodService();

        Admin admin = new Admin(1, "SystemAdmin");
        Customer customer = new Customer(2, "Rahul");

        FoodItem pizza = new FoodItem(101, "Pizza", 300);
        FoodItem burger = new FoodItem(102, "Burger", 150);

        service.addCustomer(customer);
        service.addFoodItem(pizza);
        service.addFoodItem(burger);

        Order order = new Order(1001, customer);

        order.addItem(pizza);
        order.addItem(burger);

        order.calculateTotal();

        // Apply 20% discount using Lambda
        order.applyCoupon(total -> total * 0.8);

        service.placeOrder(order);

        service.analyticsDashboard();
    }
}

```